CampusConnect 2026

AWS EC2 • LEMP Stack • MariaDB • PHP-FPM • Nginx

Project Documentation

This document presents the complete deployment workflow in the exact requested order, from EC2 instance creation to the final CampusConnect website accessed through the server's public IP address.

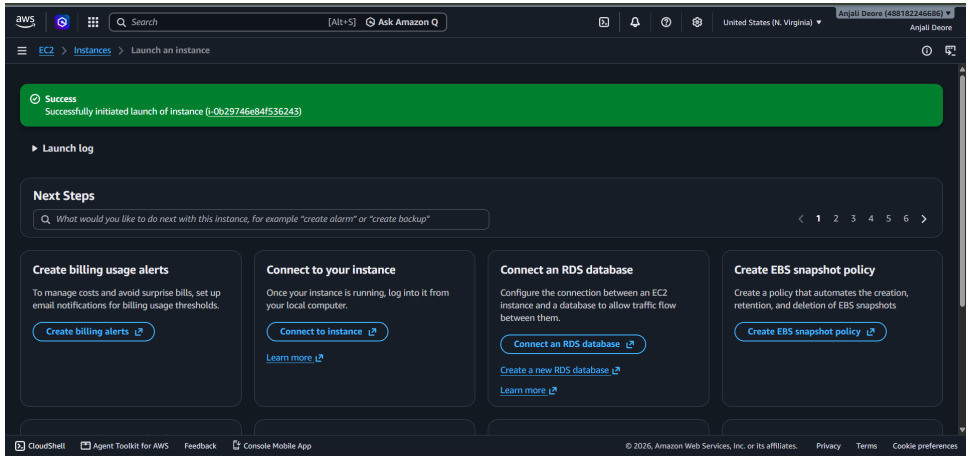
Step	Activity
1	EC2 instance launch
2	Security group – HTTP and SSH
3	Install Nginx
4	Install MariaDB
5	Install PHP
6	Start all services
7	Create database
8	Create student table
9	Create register.html
10	Create register.php
11	Registration page output
12	Create login.html

13	Create login.php
14	Login successful
15	Final website through IP

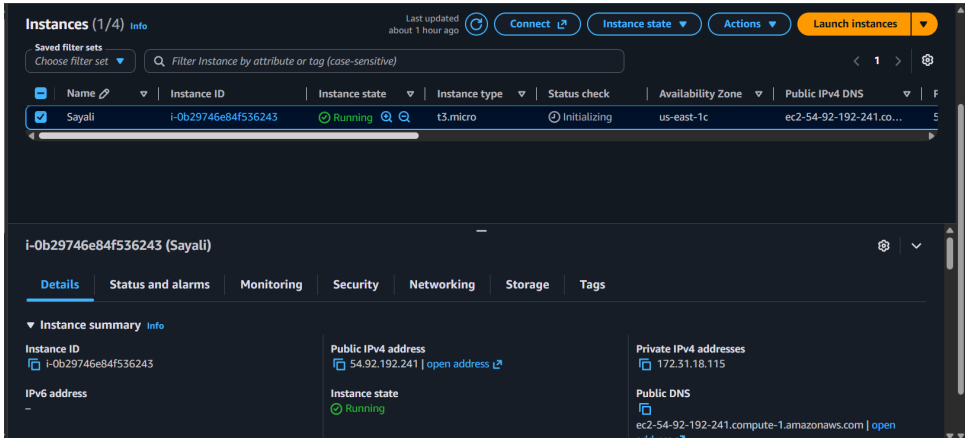
Step 1 – EC2 Instance Launch

An Amazon EC2 virtual machine was launched to host the CampusConnect 2026 application. The instance uses Amazon Linux 2023 and provides the public server environment needed for the website.

Step note: The instance is the cloud server on which Nginx, PHP-FPM and MariaDB are configured.



AWS console confirming successful EC2 instance launch

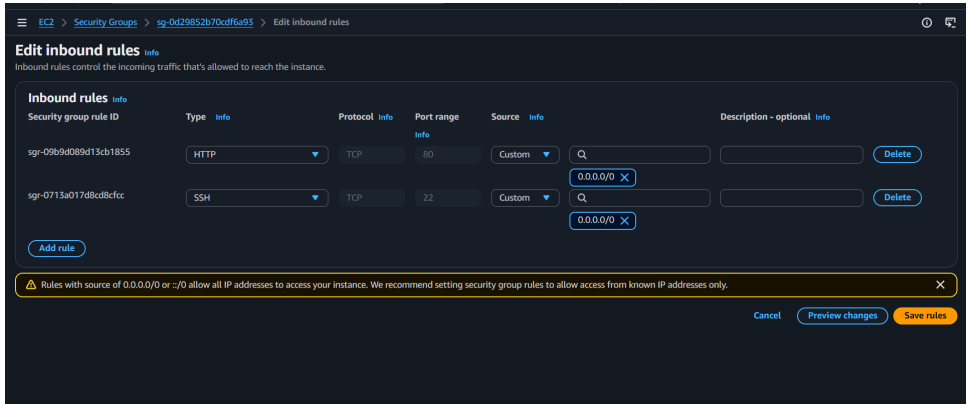


Running instance details, including the public IPv4 address.

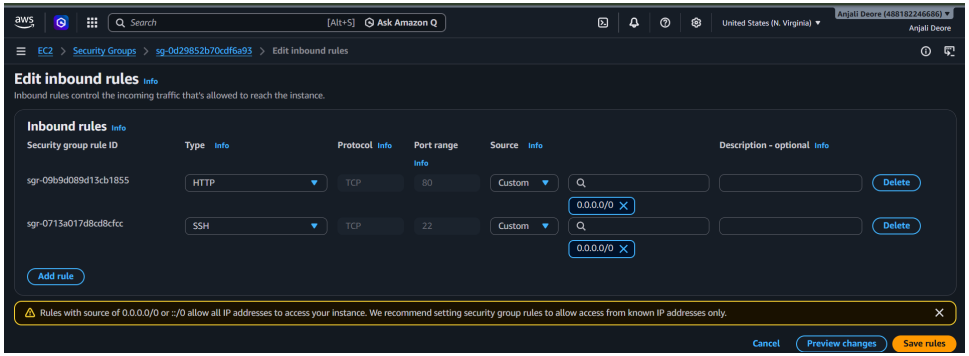
Step 2 – Security Group – HTTP and SSH

Inbound security-group rules were configured to permit HTTP traffic on port 80 for web access and SSH traffic on port 22 for remote administration of the EC2 server.

Step note: HTTP enables browser access to the website; SSH enables secure terminal access to the server.



Inbound rules showing HTTP (80) and SSH (22)

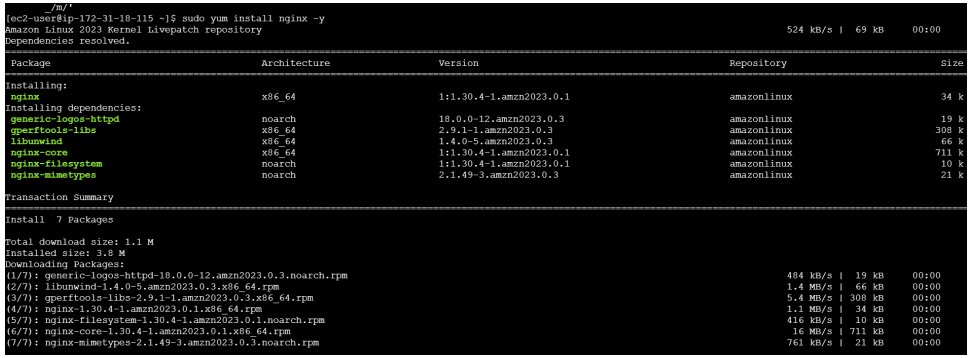


Security-group configuration in the AWS console.

Step 3 – Install Nginx

Nginx was installed as the web server. It receives HTTP requests from visitors and serves the CampusConnect pages while forwarding PHP requests to PHP-FPM.

Step note: Command shown in the screenshot: sudo yum install nginx -y



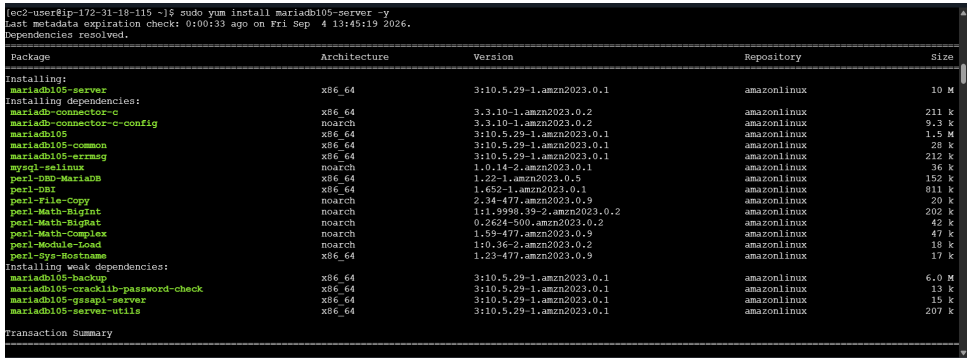
Nginx installation using the Amazon Linux package manager.

```
Command: sudo yum install nginx -y
```

Step 4 – Install MariaDB

MariaDB 10.5 was installed as the relational database server. It stores the student registration records used by the CampusConnect registration and login system.

Step note: Command shown in the screenshot: sudo yum install mariadb105-server -y



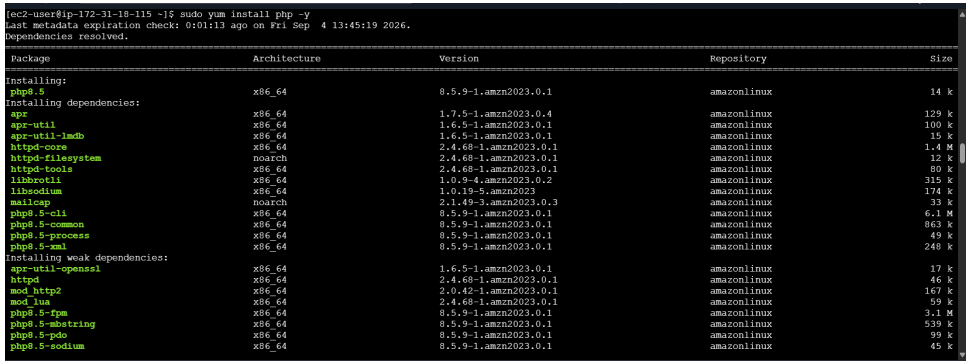
MariaDB server installation and package dependencies.

```
Command: sudo yum install mariadb105-server -y
```

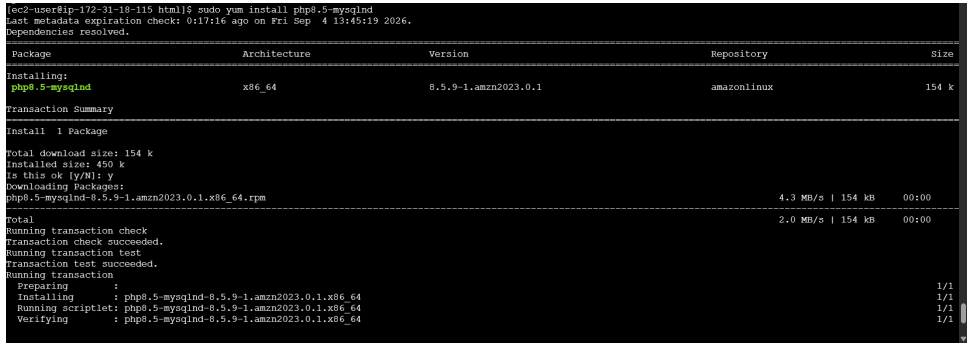
Step 5 – Install PHP

PHP was installed to execute the server-side registration and login scripts. The MySQL native driver was also installed so PHP can communicate with MariaDB.

Step note: Packages shown include PHP 8.5 and php8.5-mysqlnd.



PHP installation on Amazon Linux 2023



Installation of the PHP MySQL/MariaDB driver.

Commands: `sudo yum install php -y` | `sudo yum install php8.5-mysqlnd`

Step 6 – Start All Services

The main LEMP components were started and checked: Nginx, MariaDB and PHP-FPM. The status output confirms that the services are running on the server.

Step note: The three services work together as the LEMP application stack.

```
[ec2-user@ip-172-31-18-115 ~]$ sudo mariadb
ERROR 2002 (HY000): Can't connect to local MySQL server through socket '/var/lib/mysql/mysql.sock' (2)
[ec2-user@ip-172-31-18-115 ~]$ sudo service nginx start
Redirecting to /bin/systemctl start nginx.service
[ec2-user@ip-172-31-18-115 ~]$ sudo service mariadb start
Redirecting to /bin/systemctl start mariadb.service
[ec2-user@ip-172-31-18-115 ~]$ sudo service php-fpm start
Redirecting to /bin/systemctl start php-fpm.service
[ec2-user@ip-172-31-18-115 ~]$ sudo service nginx status
Redirecting to /bin/systemctl status nginx.service
● nginx.service - The nginx HTTP and reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
    Drop-In: /usr/lib/systemd/system/nginx.service.d
             └─php-fpm.conf
     Active: active (running) since Fri 2026-09-04 13:48:02 UTC; 48s ago
    Process: 28351 ExecStartPre=/usr/bin/rm -f /run/nginx.pid (code=exited, status=0/SUCCESS)
    Process: 28353 ExecStartPre=/usr/sbin/nginx -t (code=exited, status=0/SUCCESS)
    Process: 28354 ExecStart=/usr/sbin/nginx (code=exited, status=0/SUCCESS)
   Main PID: 28360 (nginx)
      Tasks: 3 (limit: 1059)
     Memory: 3.5M
        CPU: 78ms
     CGroup: /system.slice/nginx.service
             ├─28360 "nginx: master process /usr/sbin/nginx"
             ├─28361 "nginx: worker process"
             └─28362 "nginx: worker process"

Sep 04 13:48:02 ip-172-31-18-115.ec2.internal systemd[1]: Starting nginx.service - The nginx HTTP and reverse proxy server...
Sep 04 13:48:02 ip-172-31-18-115.ec2.internal nginx[28353]: nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
Sep 04 13:48:02 ip-172-31-18-115.ec2.internal nginx[28353]: nginx: configuration file /etc/nginx/nginx.conf test is successful
Sep 04 13:48:02 ip-172-31-18-115.ec2.internal systemd[1]: Started nginx.service - The nginx HTTP and reverse proxy server.
```

Starting Nginx, MariaDB and PHP-FPM; Nginx status is active.

```
[ec2-user@ip-172-31-18-115 ~]$ sudo service mariadb status
Redirecting to /bin/systemctl status mariadb.service
● mariadb.service - MariaDB 10.5 database server
     Loaded: loaded (/usr/lib/systemd/system/mariadb.service; disabled; preset: disabled)
     Active: active (running) since Fri 2026-09-04 13:48:20 UTC; 42s ago
       Docs: man:mariadbd(8)
             https://mariadb.com/kb/en/library/systemd/
    Process: 28431 ExecStartPre=/usr/libexec/mariadb-check-socket (code=exited, status=0/SUCCESS)
    Process: 28453 ExecStartPre=/usr/libexec/mariadb-prepare-db-dir mariadb.service (code=exited, status=0/SUCCESS)
    Process: 28550 ExecStartPost=/usr/libexec/mariadb-check-upgrade (code=exited, status=0/SUCCESS)
   Main PID: 28536 (mariadbd)
     Status: "Taking your SQL requests now..."
      Tasks: 13 (limit: 1059)
     Memory: 66.1M
        CPU: 465ms
     CGroup: /system.slice/mariadb.service
             └─28536 /usr/libexec/mariadbd --basedir=/usr

Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: The second is mysql@localhost, it has no password either, but
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: you need to be the system 'mysql' user to connect.
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: After connecting you can set the password, if you would need to be
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: able to connect as any of these users with a password and without sudo
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: See the MariaDB Knowledgebase at https://mariadb.com/kb
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: Please report any problems at https://mariadb.org/jira
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: The latest information about MariaDB is available at https://mariadb.org/.
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: Consider joining MariaDB's strong and vibrant community:
Sep 04 13:48:19 ip-172-31-18-115.ec2.internal mariadb-prepare-db-dir[28492]: https://mariadb.org/get-involved/
Sep 04 13:48:20 ip-172-31-18-115.ec2.internal systemd[1]: Started mariadb.service - MariaDB 10.5 database server.
```

MariaDB service status showing active/running.

```
[ec2-user@ip-172-31-18-115 ~]$ sudo service php-fpm status
Redirecting to /bin/systemctl status php-fpm.service
● php-fpm.service - The PHP FastCGI Process Manager
     Loaded: loaded (/usr/lib/systemd/system/php-fpm.service; disabled; preset: disabled)
     Active: active (running) since Fri 2026-09-04 13:48:02 UTC; 1min 13s ago
   Main PID: 28352 (php-fpm)
     Status: "Processes active: 0, idle: 5, Requests: 0, slow: 0, Traffic: 0.00req/sec"
      Tasks: 6 (limit: 1059)
     Memory: 11.3M
        CPU: 69ms
     CGroup: /system.slice/php-fpm.service
             ├─28352 "php-fpm: master process (/etc/php-fpm.conf)"
             ├─28355 "php-fpm: pool www"
             ├─28356 "php-fpm: pool www"
             ├─28357 "php-fpm: pool www"
             ├─28358 "php-fpm: pool www"
             └─28359 "php-fpm: pool www"

Sep 04 13:48:02 ip-172-31-18-115.ec2.internal systemd[1]: Starting php-fpm.service - The PHP FastCGI Process Manager...
Sep 04 13:48:02 ip-172-31-18-115.ec2.internal systemd[1]: Started php-fpm.service - The PHP FastCGI Process Manager.
```

PHP-FPM service status showing active/running.

```
[ec2-user@ip-172-31-18-115 html]$ sudo service nginx restart
Redirecting to /bin/systemctl restart nginx.service
[ec2-user@ip-172-31-18-115 html]$ sudo service mariadb restart
Redirecting to /bin/systemctl restart mariadb.service
[ec2-user@ip-172-31-18-115 html]$ sudo service php-fpm restart
Redirecting to /bin/systemctl restart php-fpm.service
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.html
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.php
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.php
[ec2-user@ip-172-31-18-115 html]$
```

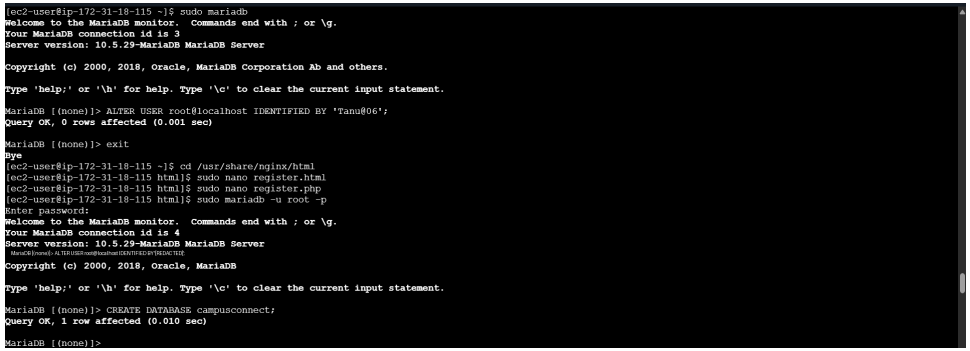
Restarting Nginx, MariaDB and PHP-FPM after configuration/file changes.

Service commands: `sudo service nginx start | sudo service mariadb start | sudo service php-fpm start`

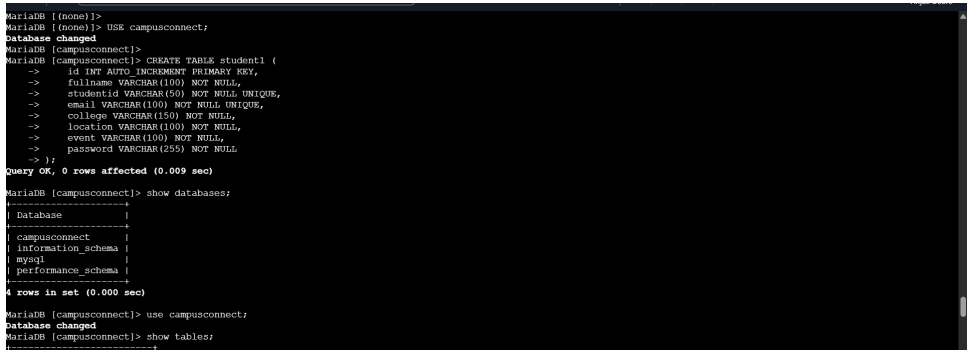
Step 7 – Create Database

A MariaDB database named campusconnect was created and selected. This database is used by the PHP scripts to store and verify student information.

Step note: Database used by the project: campusconnect



MariaDB access and database administration on the server.



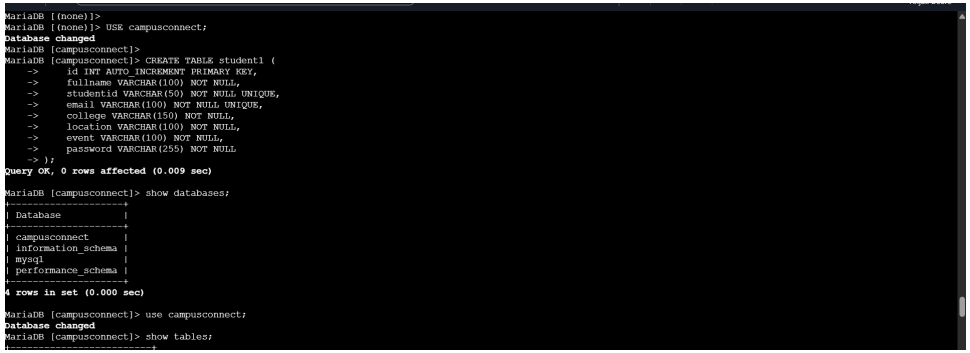
The campusconnect database is selected before creating the student table.

```
Database: campusconnect
```

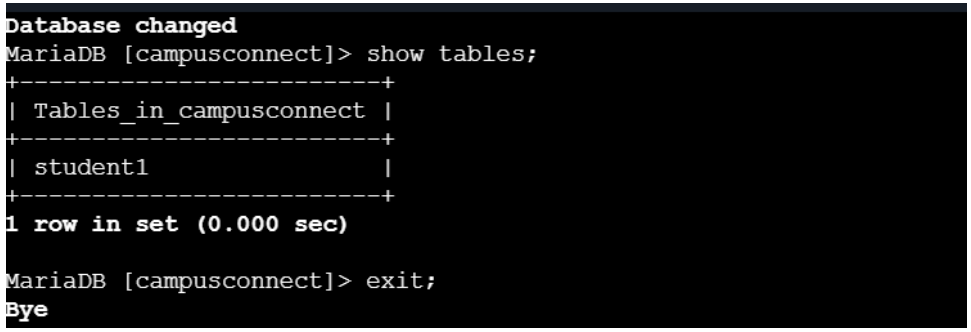
Step 8 – Create Student Table

The student1 table was created with fields for the student's name, student ID, email, college, location, event and password. The table uses an auto-increment primary key and unique student ID/email values.

Step note: The table is the database backend for both registration and login.



SQL used to create the student1 table.



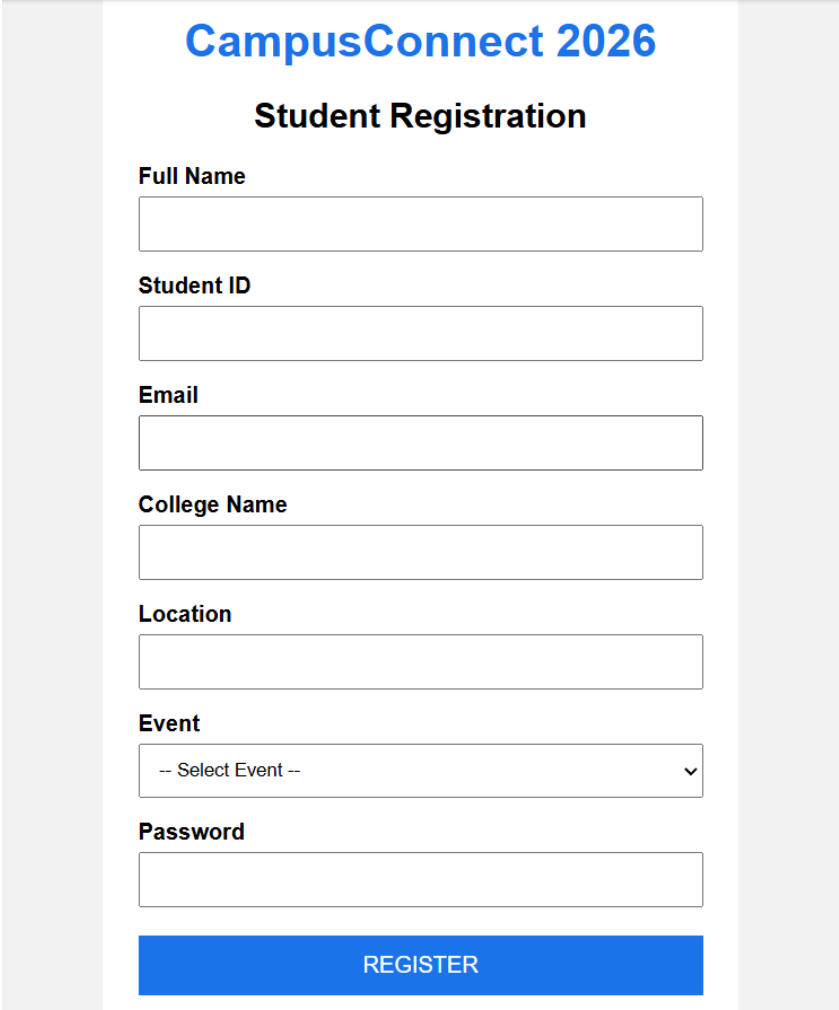
SHOW TABLES confirms that student1 exists.

Table: student1

Step 9 – Create register.html

The Student Registration page was created with all required fields: Full Name, Student ID, Email, College Name, Location, Event and Password. The form submits its data to register.php using POST.

Step note: The form provides the user interface for collecting student registration details.



CampusConnect 2026 Student Registration page showing all required fields.

```
Form target: register.php    |    Method: POST
```

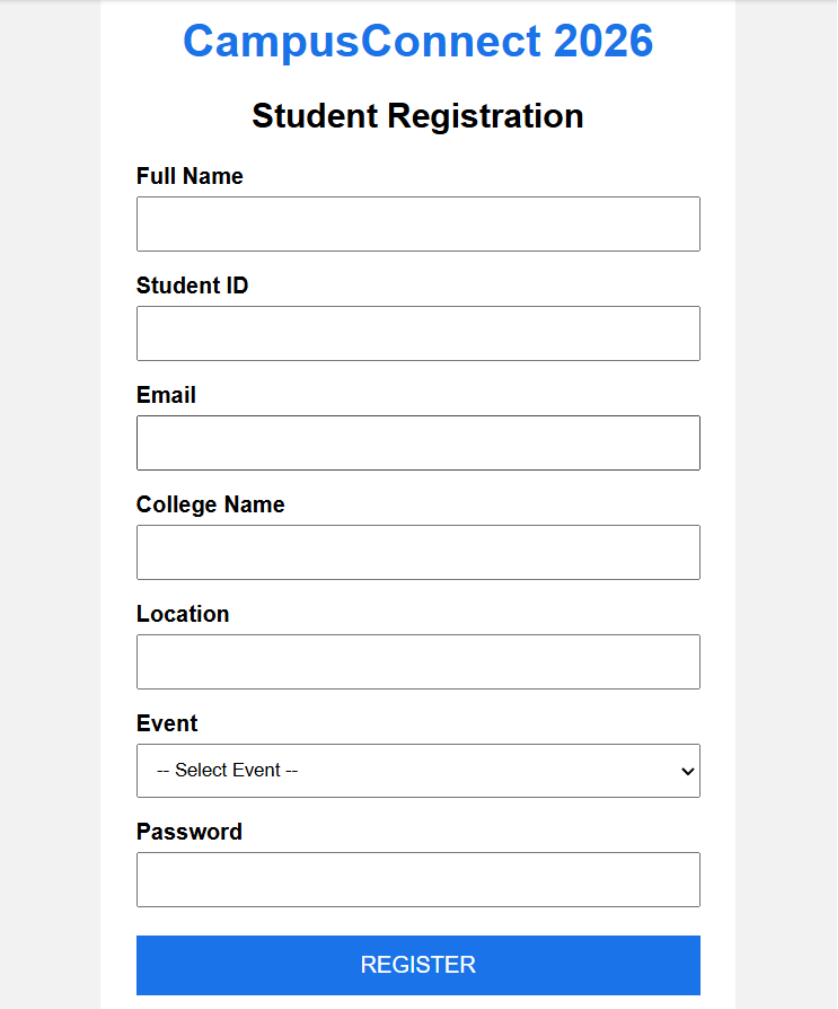
Step 10 – Create register.php

The server-side registration script was created to receive the form data, connect to the campusconnect database and insert the student details into student1. The password is processed securely before storage.

Step note: register.php is the backend endpoint specified by the registration form action.

```
[ec2-user@ip-172-31-18-115 html]$ sudo service nginx restart
Redirecting to /bin/systemctl restart nginx.service
[ec2-user@ip-172-31-18-115 html]$ sudo service mariadb restart
Redirecting to /bin/systemctl restart mariadb.service
[ec2-user@ip-172-31-18-115 html]$ sudo service php-fpm restart
Redirecting to /bin/systemctl restart php-fpm.service
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.html
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.php
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.php
[ec2-user@ip-172-31-18-115 html]$
```

Terminal workflow showing PHP/HTML file creation/editing and service restart.

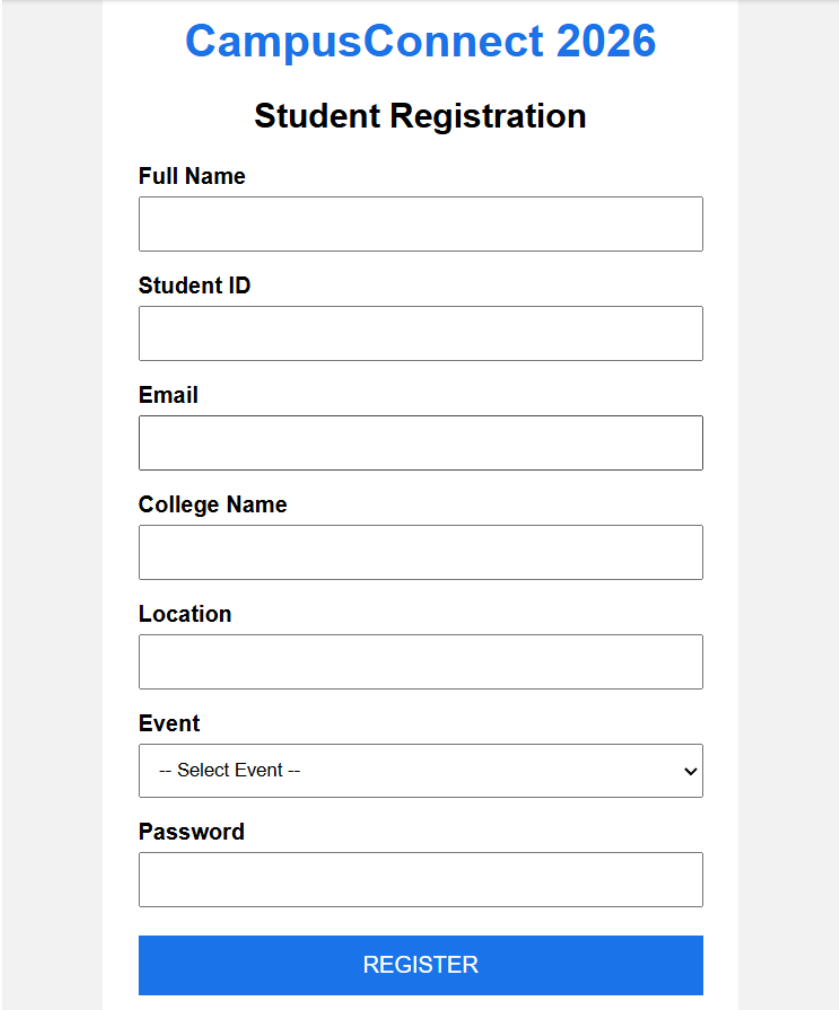


The completed registration interface is ready to submit data to register.php.

Step 11 – Registration Page Output

The deployed registration page was tested through the web server. The output displays the CampusConnect 2026 heading and all required student registration fields with a Register button.

Step note: This confirms that the HTML registration interface is accessible through the deployed web environment.

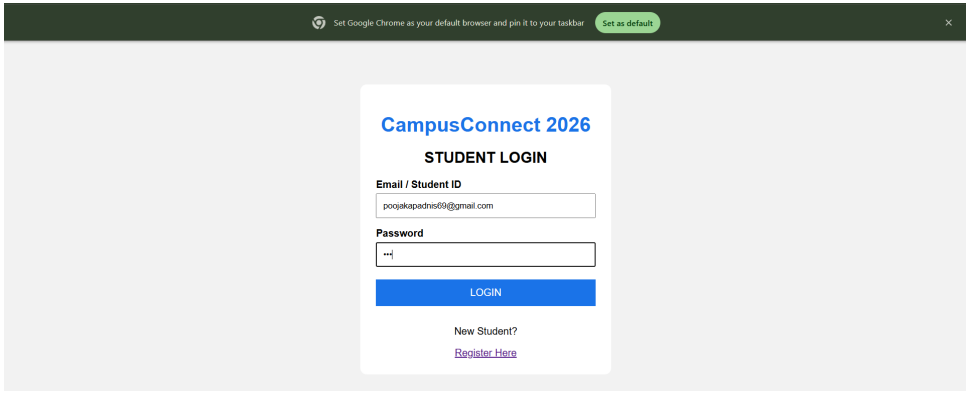


Final registration page displayed by the server.

Step 12 – Create login.html

A separate Student Login page was created. It contains the required Email / Student ID field and Password field, with a Login button and a link back to registration.

Step note: The form sends the entered credentials to login.php using POST.



CampusConnect 2026 Student Login page.

```
Form target: login.php    |    Method: POST
```

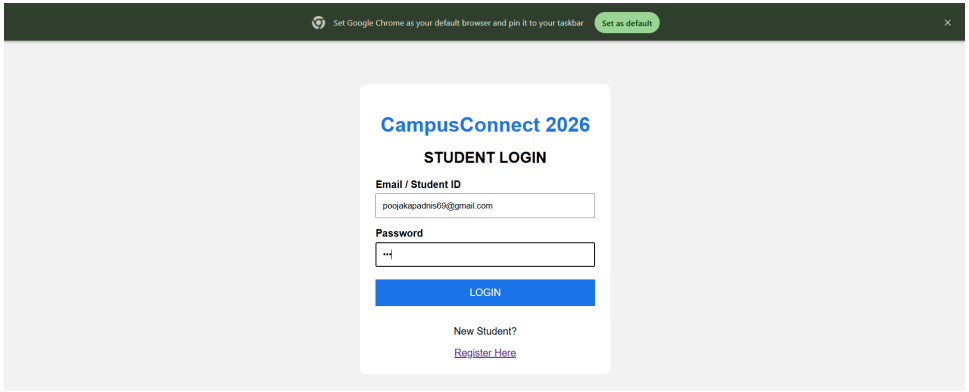
Step 13 – Create login.php

The login backend was created to query student1 using either the email address or student ID. It then verifies the submitted password against the stored password value.

Step note: login.php implements the database-backed credential verification required by the task.

```
[ec2-user@ip-172-31-18-115 html]$ sudo service nginx restart
Redirecting to /bin/systemctl restart nginx.service
[ec2-user@ip-172-31-18-115 html]$ sudo service mariadb restart
Redirecting to /bin/systemctl restart mariadb.service
[ec2-user@ip-172-31-18-115 html]$ sudo service php-fpm restart
Redirecting to /bin/systemctl restart php-fpm.service
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.html
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.php
[ec2-user@ip-172-31-18-115 html]$ sudo nano login.php
[ec2-user@ip-172-31-18-115 html]$
```

Terminal showing login.html/login.php file editing and service restart.

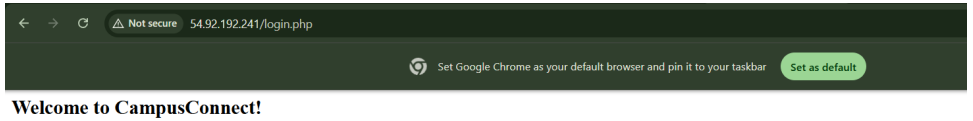


Login form that submits credentials to login.php.

Step 14 – Login Successful

After valid credentials are supplied, the login system verifies the student and displays the required success message: **Welcome to CampusConnect!**

Step note: This confirms that the PHP login flow is working and can return the required successful-login response.

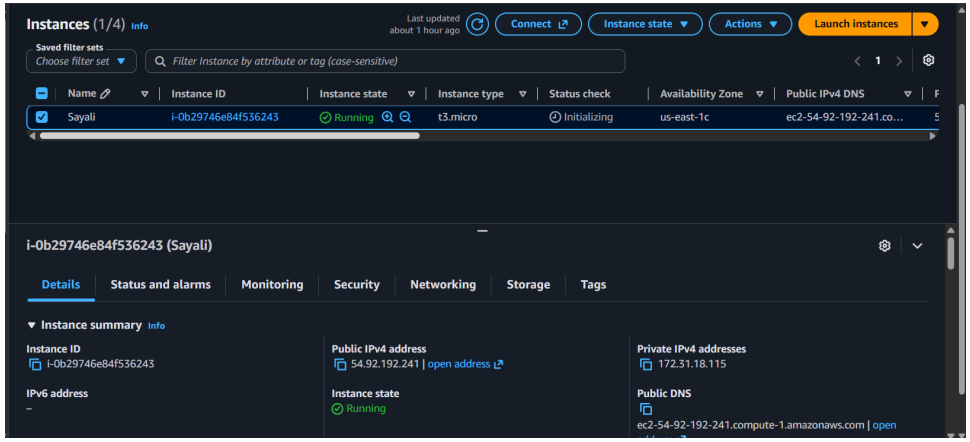


Browser output showing the successful CampusConnect login message.

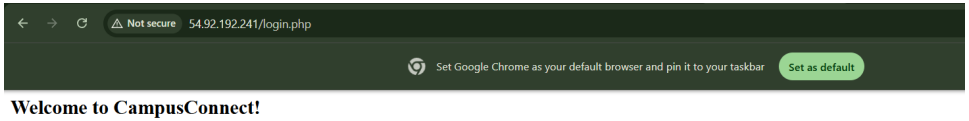
Step 15 – Final Website Through IP

The CampusConnect 2026 application is deployed on the EC2 server and is reachable through the server's public IP address. The final browser output demonstrates that the application is being served from the cloud instance.

Step note: Final public address used in the screenshots: http://54.92.192.241/



EC2 instance details showing public IPv4 address 54.92.192.241.



CampusConnect success output accessed through the public IP address.

Public IP URL: http://54.92.192.241/